CS310 - Mobile Application Development

Project Feedback Report

Group: Group 19 **Evaluator:** Berke Ucvet **Date:** 2025-12-28 **GitHub Repository:** https://github.com/idilsunar/CS310-Project

---

Phase 2.2: UI Implementation

Grading Summary (25 pts)

Criteria	Description	Max	Score
Visual Consistency	UI matches wireframe, visually cohesive	7	Complete
Code Structure	Proper widget hierarchy, naming, organization	5	Complete
Navigation	Named routes and page transitions work correctly	5	Partial
Responsiveness	Works well on various screen sizes	5	Complete
Repository & Submission	Clear commits, repo organization, documentation	3	Complete
Total		**25**	

Technical Requirements Checklist

Requirement	Status	Notes
Named Routes (initial route, transitions)	[ ]	Uses MaterialPageRoute, not named routes. No `routes:` map or `Navigator.pushNamed` found
Utility Classes (colors, text styles, paddings)	[x]	`lib/utils/app_colors.dart` and `lib/utils/app_text_styles.dart` well-implemented
Asset Images (local)	[x]	`Image.asset('assets/images/logo.png')` in `home_screen.dart:345`
Network Images (URL)	[ ]	No `Image.network` usage found in codebase
Custom Font (via theme)	[x]	Declared in `pubspec.yaml` (AppFont with RobotoCondensed-Regular.ttf). However, not applied via ThemeData fontFamily
Card + List Component (from model class)	[x]	`ListView.builder` with Card-like containers for habits in `home_screen.dart:562-690`, uses `Habit` model class
Dynamic Remove Action	[x]	`onLongPress` calls `habitProvider.deleteHabit(habit.id)` in `home_screen.dart:611`
Form Screen (2+ input fields)	[x]	`add_habit_screen.dart` has Form with 4 fields (name, category, frequency, color)

Requirement	Status	Notes
Inline Validation	[x]	TextFormField with `validator:` functions in `add_habit_screen.dart: 45-46, 66, 85, 106`
AlertDialog on Success	[ ]	AlertDialog only shown on form error (`add_habit_screen.dart :136-143`), not on success
Responsiveness	[x]	Uses `Expanded`, `Flexible`, `SingleChildScrollView`, `ConstrainedBox`. No `MediaQuery` or `LayoutBuilder`
Project Runs Without Errors	[x]	Project structure is correct, dependencies properly configured

Feedback

Strengths:

- Excellent code organization with clear separation into `screens/`, `providers/`, `repositories/`, `models/`, and `utils/` directories
- Well-implemented utility classes with comprehensive color palette and text style constants
- Clean ListView implementation with attractive card-styled habit items
- Form validation is properly implemented with validators on all input fields

Areas for Improvement:

- **Named Routes Missing (-1 pt):** The app uses `MaterialPageRoute` and `Navigator.push()` throughout. To meet requirements, implement named routes in `main.dart` with a `routes:` map and use `Navigator.pushNamed()` for navigation
- **Network Images Missing:** Add at least one `Image.network()` usage (e.g., user avatar from URL, or a placeholder image service)

- **Custom Font Not Applied in Theme:** While the font is declared in `pubspec.yaml`, it should be applied in the `ThemeData` via `fontFamily: 'AppFont'` for consistent usage
- **AlertDialog on Success:** Currently shows dialog only on form error. Add an AlertDialog confirming successful habit creation before navigating back
- **Consider Adding MediaQuery:** For better responsiveness, use `MediaQuery.of(context).size` to adapt layouts to different screen sizes

---

Phase 3: Firebase Backend & State Management

Grading Summary (24 pts)

#	Criteria	Max	Score
1	Firebase setup and connection	2	Complete
2	Firebase Auth (sign up, log in, log out)	3	Complete
3	Firestore data model (well-organized collections)	2	Complete
4	Dart model classes represent Firestore documents	2	Complete
5	Service/repository layer (no direct Firestore in UI)	2	Complete
6	Provider setup (MultiProvider, ChangeNotifier)	2	Complete
7	Auth state management via provider	2	Complete

#	Criteria	Max	Score
8	Loading/success/error states (StreamBuilder/FutureBuilder)	2	Complete
9	Real-time UI updates when Firestore data changes	2	Complete
10	Navigation & access control (protected routes)	2	Complete
11	SharedPreferences (at least one preference)	2	Complete
12	Firestore Security Rules deployed	2	Critical
	Total	**24**	

Requirements Checklist

Firebase Authentication

- ☑ User sign-up (email & password) - `auth_repository.dart:9-40`
- ☑ User login - `auth_repository.dart:42-54`
- ☑ User logout - `auth_repository.dart:56-62`
- ☑ Error handling with user-friendly messages - `auth_repository.dart:72-91` maps Firebase error codes to user-friendly text

Cloud Firestore

- ☑ Documents include: id, app-specific fields, createdBy, createdAt - `habit.dart` model has all required fields
- ☑ Create operation - `habit_repository.dart:61-69`
- ☑ Read operation - `habit_repository.dart:72-86` returns Stream
- ☑ Update operation - `habit_repository.dart:88-94`
- ☑ Delete operation - `habit_repository.dart:96-102`

- ☑ Real-time updates (streams) - Uses `.snapshots()` in `habit_repository.dart:74-82`
- ☐ Security rules prevent unauthenticated access - No `firestore.rules` file found in repository (If your video demo shows the rules I can adjust this score)

State Management (Provider)

- ☑ MultiProvider configured - `main.dart:29-36` with 5 providers
- ☑ ChangeNotifier classes implemented - All providers extend `ChangeNotifier`
- ☑ Auth state managed via provider - `auth_provider.dart` with `listenToAuthState()`
- ☑ App reacts to login/logout - `auth_wrapper.dart` uses Consumer to navigate based on auth state
- ☑ App reacts to Firestore data changes - `HabitProvider` uses stream subscription, UI updates automatically

Local Persistence

- ☑ SharedPreferences saves at least one preference - `preferences_provider.dart` saves isDarkMode, hasCompletedOnboarding, lastSelectedTab, dailyNote
- ☑ Preference restored on app restart - `loadPreferences()` called in constructor

Demo Video

- ☐ Within 5 minutes - Not evaluated
- ☐ Shows authentication flow - Not evaluated
- ☐ Shows CRUD operations - Not evaluated
- ☐ Shows real-time updates - Not evaluated

Feedback

Strengths:

- **Excellent Architecture:** The repository pattern is well-implemented with clean separation between UI, providers, and data layer. This is professional-level code organization
- **Comprehensive Auth Implementation:** Error handling in `auth_repository.dart` properly maps Firebase error codes to user-friendly messages (weak-password, email-already-in-use, etc.)
- **Rich Model Classes:** `Habit`, `HabitCompletion`, and `Reminder` models have proper `fromFirestore`/`toFirestore` methods with null safety handling

- **Real-time Updates:** StreamBuilder in `calendar_screen.dart:346` properly handles Firestore streams with loading and error states
- **Robust Provider Setup:** 5 well-structured providers (Auth, Habit, Preferences, Achievements, Reminder) all properly extending ChangeNotifier
- **Comprehensive SharedPreferences:** Stores multiple preferences including dark mode, onboarding status, last tab, and daily notes

Areas for Improvement:

- **Firestore Security Rules (-2 pts):** No `firestore.rules` file found in the repository. Security rules are critical for production apps. Add rules like:

```
None
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /habits/{habitId} {
      allow read, write: if request.auth != null && request.auth.uid ==
resource.data.createdBy;
    }
  }
}
```

- **Consider Adding firebase_options.dart:** While Firebase initializes without explicit options, using `flutterfire configure` generates platform-specific configuration for more reliable setup

---

Overall Summary

Strengths

1. **Outstanding Code Organization:** The project demonstrates excellent architectural decisions with clear separation of concerns (screens, providers, repositories, models, utils). This structure is maintainable and scalable.

2. **Comprehensive State Management:** The Provider implementation is thorough with proper loading states, error handling, and

`notifyListeners()` calls throughout. The `AuthProvider` correctly listens to auth state changes.

3. **Well-Designed Data Layer:** The repository pattern with Firestore abstraction is clean. Model classes have proper serialization methods and handle null values gracefully.

4. **Rich Feature Set:** The app includes many features beyond requirements: dark mode toggle, onboarding flow, achievements system, calendar view with real-time habit tracking, and daily notes.

5. **User Experience:** Good attention to UX with loading indicators, error messages, and visual feedback (color-coded habit status, animations).

Areas for Improvement

1. **Named Routes (Critical):** Implement proper named routing in `main.dart` to meet Phase 2.2 requirements. This is a fundamental Flutter navigation pattern.

2. **Network Images:** Add at least one network image to satisfy Phase 2.2 requirements.

3. **Firestore Security Rules:** This is a critical security concern. Without rules, any user can read/write any data. Deploy proper security rules to Firebase Console.

4. **AlertDialog on Success:** Add confirmation dialogs when forms are successfully submitted, not just on errors.

5. **Theme Font Application:** Apply the custom font in ThemeData for consistent typography across the app.

Additional Notes

- The `STEP3_TODO.md` file in the project suggests good planning practices.
- Code quality is high with proper widget decomposition and reusable components.
- The achievements and calendar features show ambition beyond basic requirements.
- Consider adding unit tests for repositories and providers to improve code reliability.

Evaluator's Notes

Repository Structure

- The repo structure is good but the repo itself is another folder in the repo. **Get rid of the wrapping folder** and make sure that all your content is in the root of the repository.

Firebase Platform Support (Critical)

- The app **does not support Firebase in the web version**. It needs to be supported.
- The app **does not support Firebase in the iOS version**. It needs to be supported.
- Run `flutterfire configure` to generate proper platform configurations.

Authentication Flow Issues (Critical)

- **Signup should not redirect to login.** After successful signup, the user should be taken directly into the app. You are making users go through extra steps which increases the chance they will abandon the app.
- **Login state bug:** After successful login (snackbar shows "Login successful"), the app incorrectly returns to the onboarding screens. Completing onboarding again and trying to login causes the same loop. Only a hot restart resolves this. This is most likely a **state management/navigation issue** in `AuthWrapper` or `PreferencesProvider`. This must be addressed.

UX Improvements

- The **"Note for today" section is confusing.** Its purpose is not immediately clear to users. Consider adding a placeholder text or tooltip explaining what it's for.
- The **Calendar screen could be an actual calendar grid** instead of listing all days in a month. A traditional calendar view would provide better UX and quicker navigation.
- Both **Calendar and Settings pages need to be wrapped in a SafeArea widget** to properly handle device notches and system UI.

Overall Assessment

Very good work. Almost all requirements are met. There is room for improvement only in small places. The architecture and code quality are strong - focus on fixing the critical auth flow bug and Firebase platform support to make this a polished application.